Diagramas de Interacción

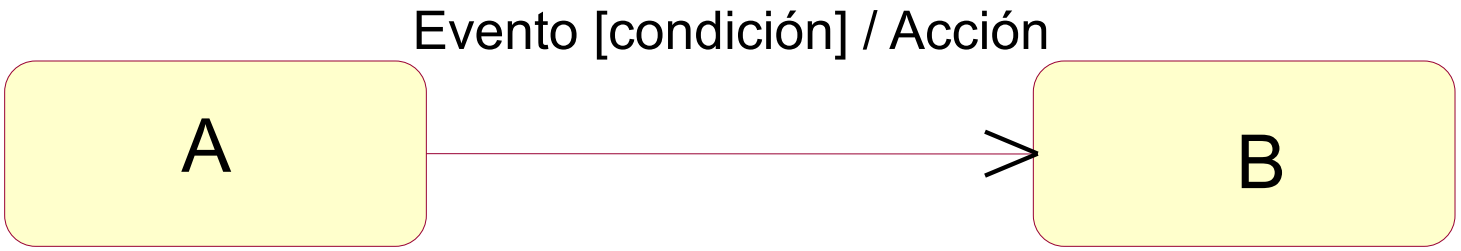
- **Diagramas de Secuencia vs. Diagramas de Colaboración**

Tipo	*Puntos fuertes*	*Puntos débiles*
secuencia	muestra claramente la secuencia u ordenación en el tiempo de los mensajes notación simple	fuerza a extender por la derecha cuando se añaden nuevos objetos; consume espacio horizontal
colaboración	economiza espacio, flexibilidad al añadir nuevos objetos en dos dimensiones es mejor para ilustrar bifurcaciones complejas, iteraciones y comportamiento concurrente	difícil ver la secuencia de mensajes notación más compleja

Diagrama de Estados

… Diagrama de Estados

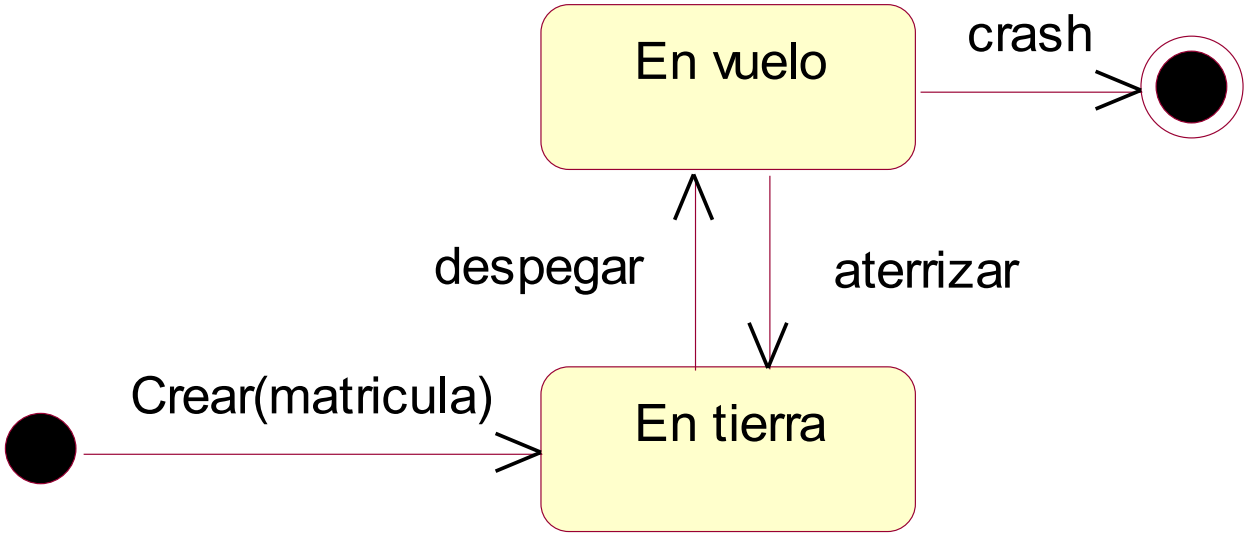
- **Estados y Transiciones**



Tanto el evento como la acción se consideran instantáneos

… Destrucción de Objeto

- **Ejemplo:**



… Transiciones temporizadas

- **Ejemplo:**

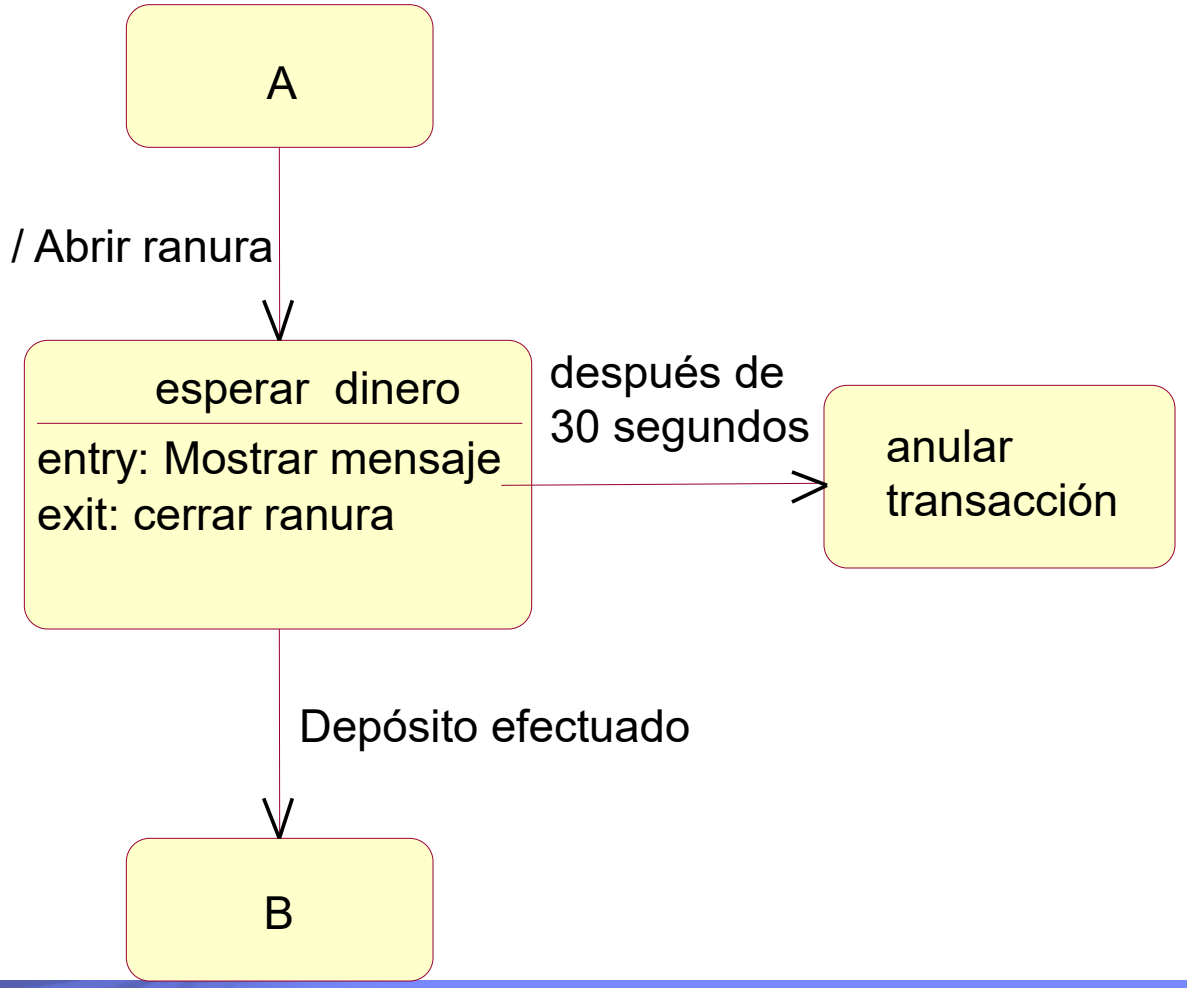


Diagrama de Componentes

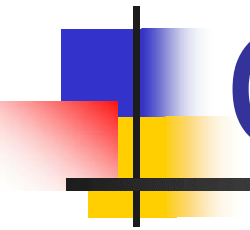


Diagrama de Componentes

- El Diagrama de Componentes **ilustra las piezas de software que componen el sistema.**
- Tiene un **mayor nivel de abstracción que un Diagrama de Clases:** Usualmente un componente es implementada por una o mas clases (u objetos) en tiempo de ejecución (runtime).
- Un **componente puede englobar una importante porción del sistema.**

Diagrama de Componentes

- Ejemplo:
- Relaciones:

1) **Assembly:**
componente brinda una **interfaz**
y otro componente la necesita.
Ej. Orden hacia Producto y Cliente
2) **Dependencia:** A veces explicitamos
que brinda un **componente** a otro.
Ej. Cuenta y Orden
3) **Asociacion:** Muestra que un
componente esta relacionado con ot
como Account y Customer

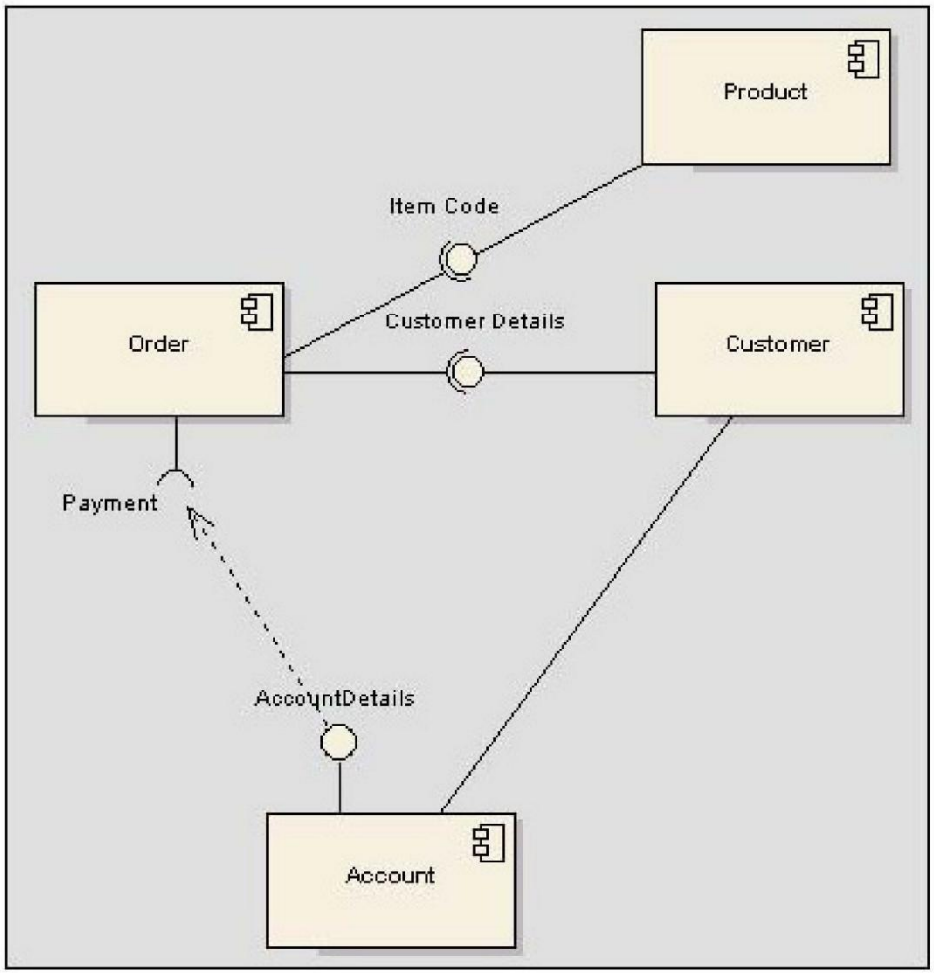


Diagrama de Componentes

- Como crear un Diagrama de Componentes:

1) Definir **que componentes** tiene el sistema.
2) Establecer **que relaciones** define ese componente con otros:

- ¿Qué información **necesita de otros componentes**?
- ¿Qué información **brinda a otros componentes**?

Diagramas de Componentes

- Cuándo utilizar diagramas de Componentes:
  - Para diseñar **sistemas a gran escala**, de modo de comprender las dependencias de los elementos principales del sistema.

Diagrama de Despliegue: Deployment

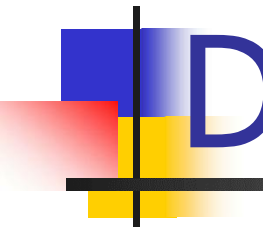


Diagrama de Despliegue

- **Modelan los aspectos físicos de los sistemas OO.**
- **Muestran las relaciones físicas entre los componentes de software y de hardware en el sistema.**
- **Marcan qué piezas de software corren en qué piezas de hardware, en un sistema distribuido.**
- **El hardware puede ser un dispositivo o un sensor simple, o puede tratarse de un mainframe.**

Diagrama de Despliegue

- Los nodos contienen artefactos, que son manifestaciones físicas de software, usualmente archivos.

- Éstos archivos deben ser:
  - Ejecutables (archivos .exe, binarios, DLLs, archivos JAR o scripts).
  - Archivos de datos
  - Archivos de configuración
  - Documentos HTML
  -

Diagrama de Despliegue

- **Ejemplo 1:**

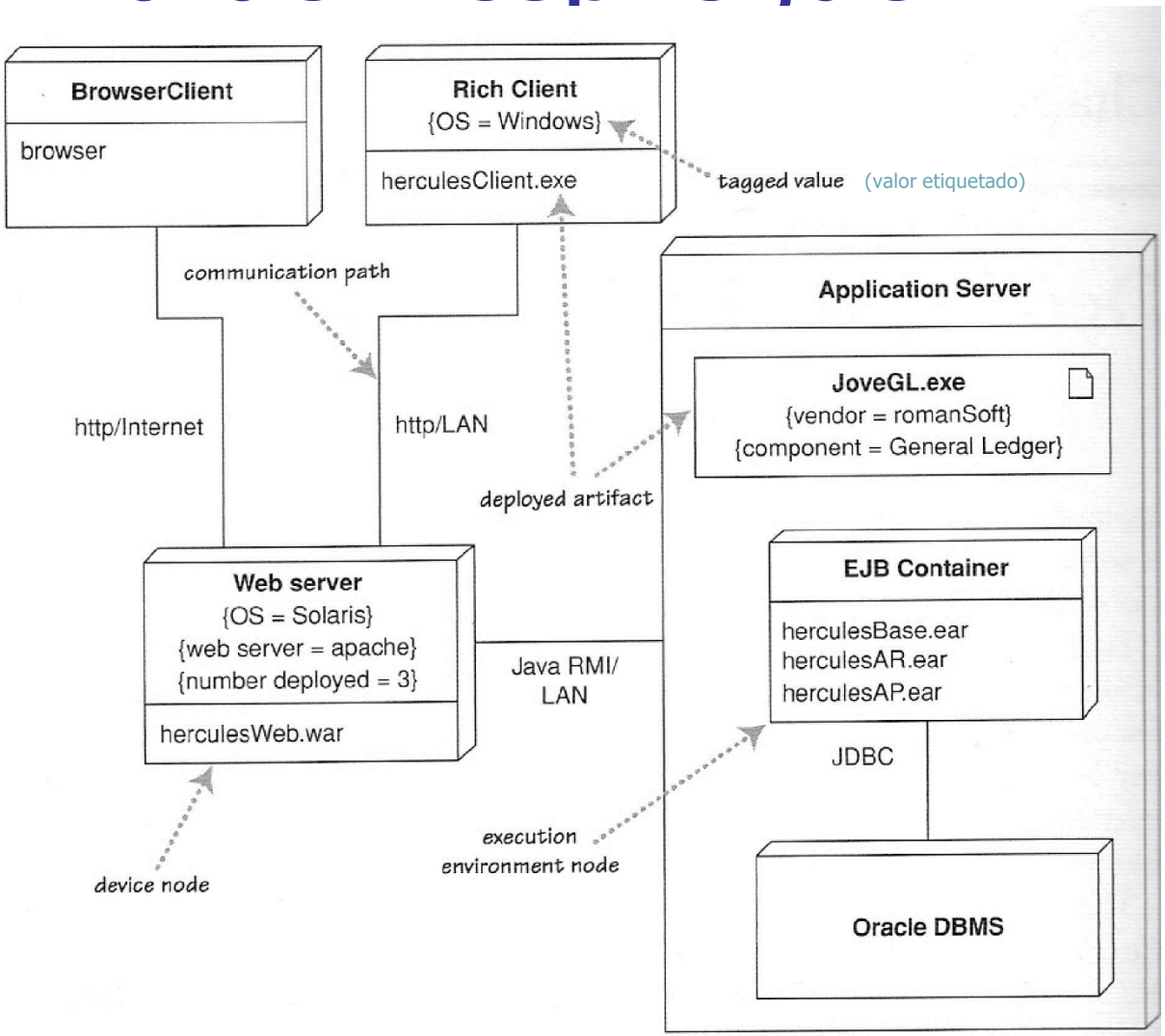


Diagrama de Despliegue

- **Cuándo utilizar Diagramas de Despliegue:**
  - **Para modelar sistemas completamente distribuidos.**

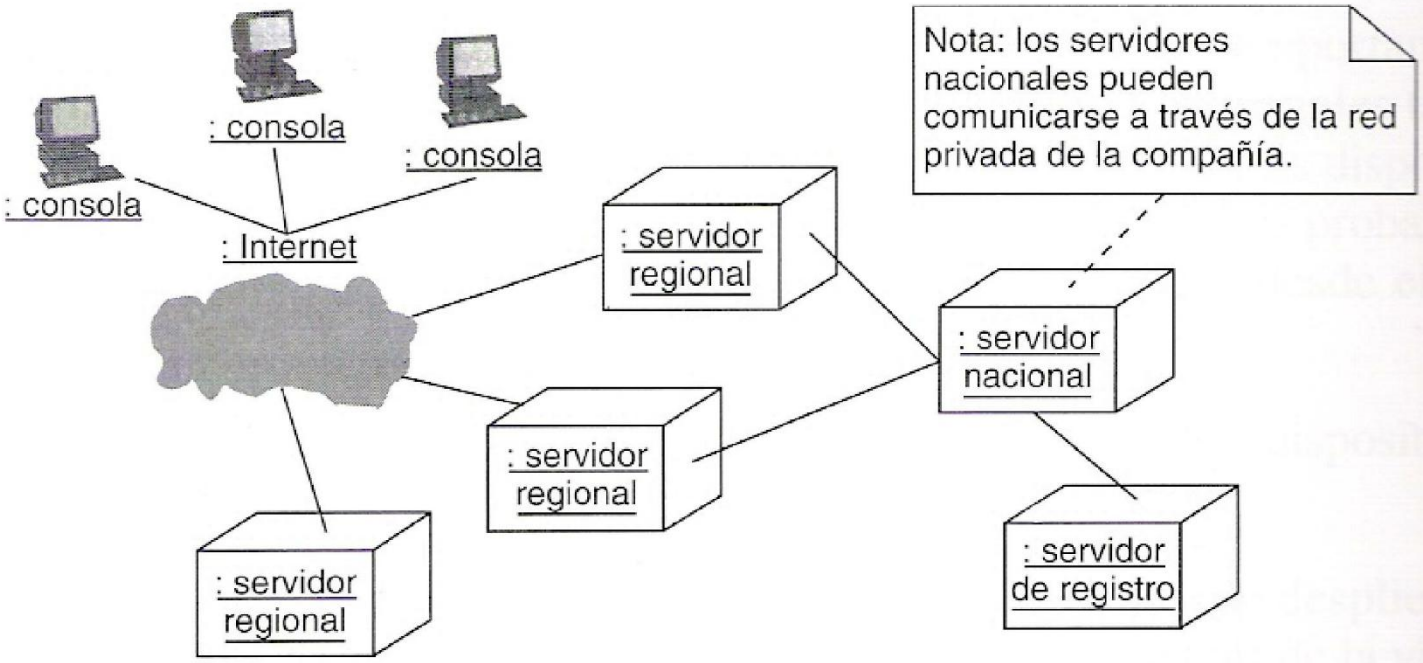
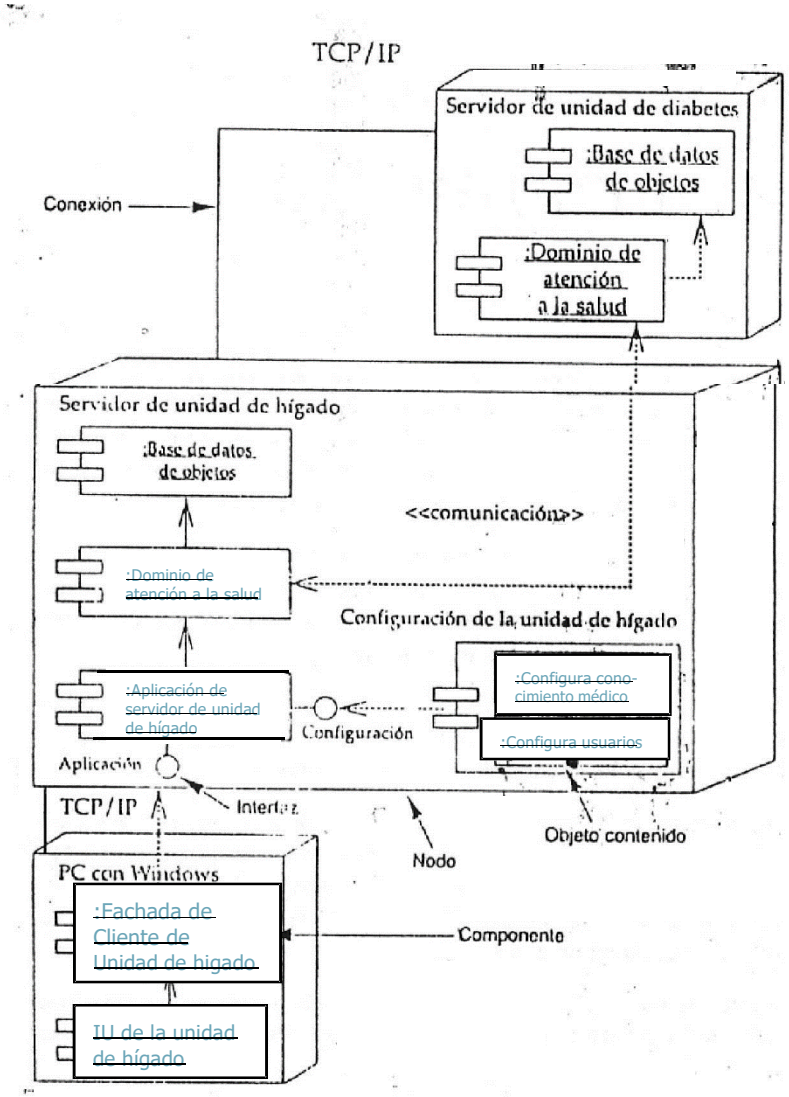


Diagrama de Despliegue

- **Ejemplo 2:**



Patrones GRASP

Repaso de conceptos: Patrones.

PATRONES: Es una **descripción** de un **problema** y su **solución** que recibe un nombre y que puede emplearse en otros contextos, con una sugerencia sobre la manera de usarlo en situaciones nuevas.

No suelen contener ideas nuevas*:* Por el contrario, intentan codificar el conocimiento, las expresiones y los principios ya existentes y trillados.

Los patrones tienen nombre*: A*signar un nombre sugestivo a un patrón ofrece las siguientes ventajas:

- Apoya el agrupamiento y la incorporación del concepto a nuestro sistema cognitivo y la memoria.
- Facilita la comunicación

Patrones GRASP

GRASP: *General Responsability Asignment Software Patterns*

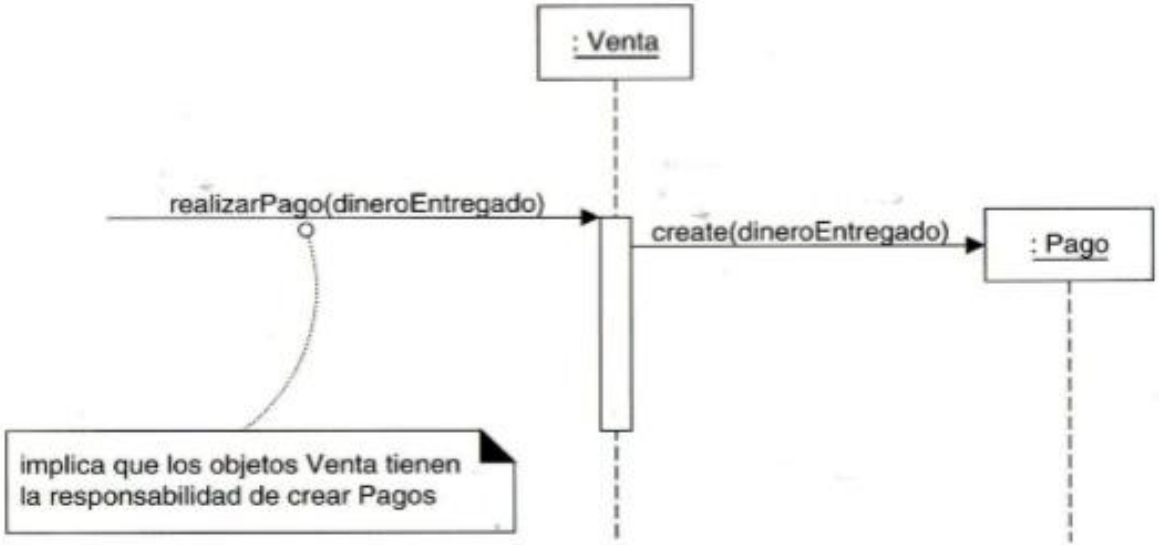
Patrones de Principios Generales Para Asignar Responsabilidades.

- Se aplican con los Diagramas de Interacción (secuencia y comunicación)

Patrones GRASP. Responsabilidades y Métodos

Responsabilidad

- Las responsabilidades y los métodos están relacionados



Patrones GRASP, responsabilidades

Responsabilidad

- Contrato u obligación de un tipo o clase
- Las responsabilidades se asignan a los objetos durante el **diseño**.

Las responsabilidades se dividen en:

- Responsabilidad de **Conocer**.
- Responsabilidad de **Hacer**.

Patrones GRASP. Responsabilidades de Conocer

Responsabilidades de un objeto relacionadas con **conocer:**

- Estar enterado de los datos privados encapsulados.
- Estar enterado de la existencia de objetos conexos.
- Estar enterado de cosas que se pueden calcular o derivar

Patrones GRASP. Responsabilidades de Hacer

Responsabilidades de un objeto relacionadas con Hacer:

- Hacer algo uno mismo.
- Iniciar una acción en otros objetos
- Controlar y coordinar actividades en otros objetos

Patrones GRASP más comunes

- **Experto**
- **Creador**
- **Alta Cohesión**
- **Bajo Acoplamiento**
- **Controlador**

Patrones GRASP: *"Experto"*

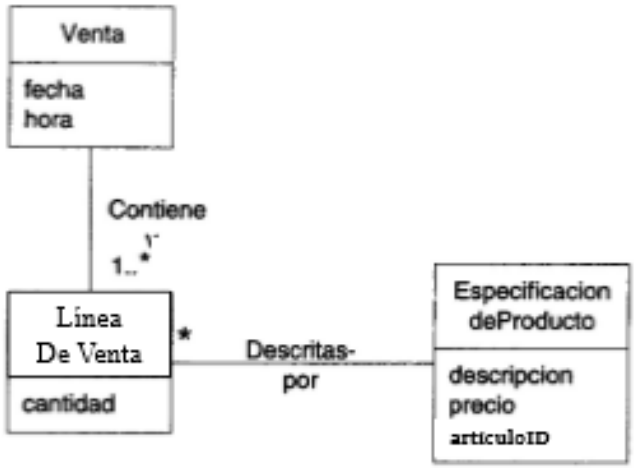
Problema: ¿Cuál es el principio para asignar responsabilidades a los objetos?

Solución: Asignar una responsabilidad al **experto en información**, manteniendo el encapsulamiento .

La clase cuenta con la información necesaria para cumplir la responsabilidad

Ejemplo PDV (Larman)

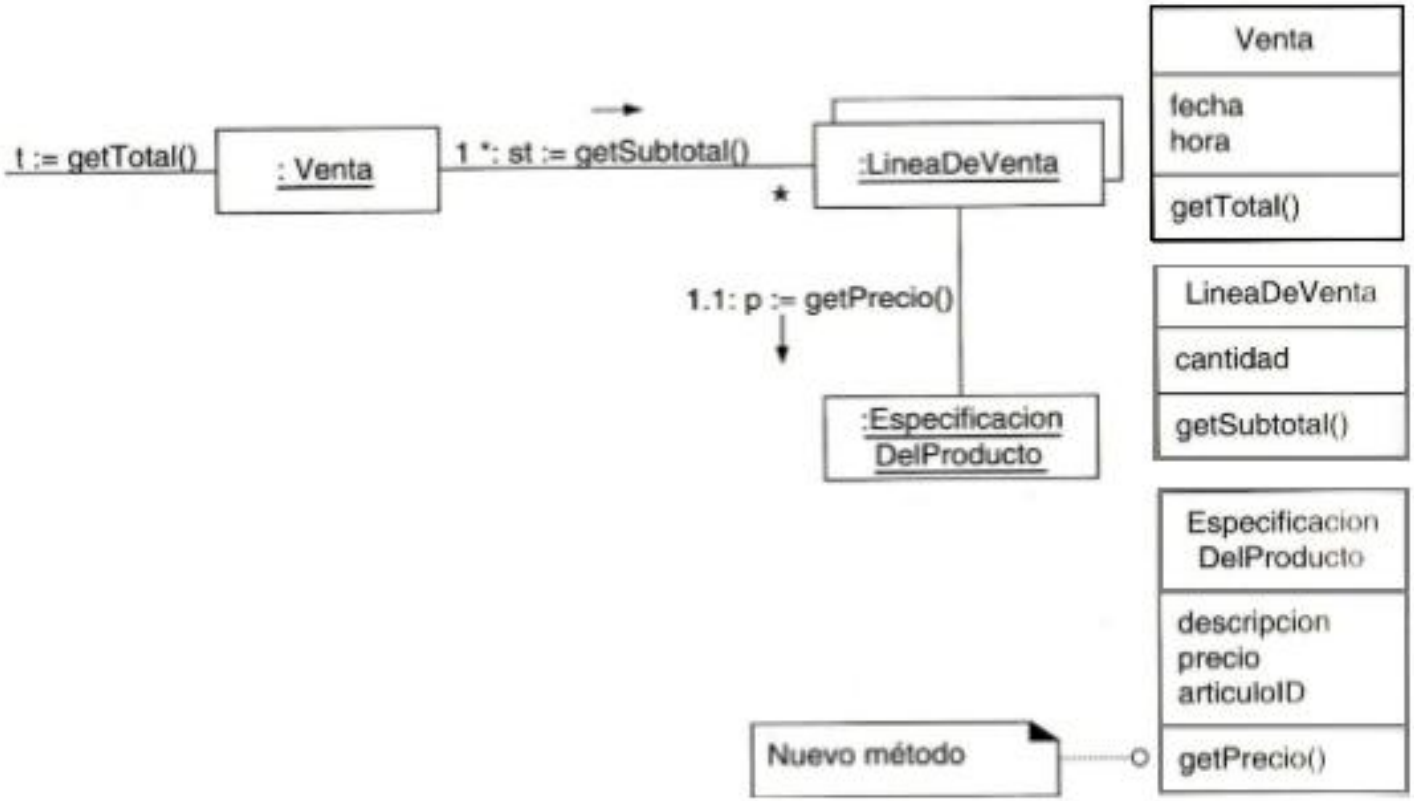
¿Quién es el responsable de conocer el total de una venta?



Patrones GRASP: *"Experto"*

Ejemplo PDV (Larman)

¿Qué información hace falta para calcular el total?



Patrones GRASP: “Experto”

- La creación de un objeto o la implementación de un método, debe **recaer sobre la clase que conoce toda la información**

- Nótese que el cumplimiento de una responsabilidad requiere a menudo información distribuida en varias clases de objetos (hay muchos **expertos “parciales”** que colaboran en la tarea).

- El patrón experto ofrece una analogía con el mundo real. Acostumbramos a asignar responsabilidades a individuos que disponen de la información necesaria para llevar a acabo una tarea.

Patrones GRASP: “Experto”

- Otras formas de designar el patrón
  - “Juntar responsabilidad y la información”
  - “Lo hace el que conoce”
  - “Lo hago yo mismo”
  - “Unir los servicios a los atributos sobre los que operan”
- Beneficios
  - Se conserva el **encapsulamiento**, ya que los objetos se valen de su **propia información** para hacer lo que se les pide. Esto soporta un **Bajo Acoplamiento**, lo que favorece el hecho de tener sistemas más robustos y de fácil mantenimiento.
  - El **comportamiento** se distribuye entre las clases que cuentan con la información requerida, **alentando** con ello definiciones de **clases** “**sencillas**” y más cohesivas que son más fáciles de comprender y de mantener. Así se brinda soporte a una **Alta Cohesión**.

Patrones GRASP: "*Creador*"

Problema

¿Quién es responsable de crear una nueva instancia de alguna clase?

Solución

Asignarle a la clase B la responsabilidad de crear una instancia de clase A en uno de los siguientes casos:

- B conoce toda la información para crear el objeto A
- B "agrega" objetos A.
- B "contiene" objetos A.
- B "registra" las instancias de los objetos de A.
- B "utiliza" estrechamente los objetos de A.
- B "tiene los datos de inicialización" que pasarán a A cuando sea creado (por tanto, B es un Experto con respecto a la creación de A).
- B es un creador de los objetos de A.

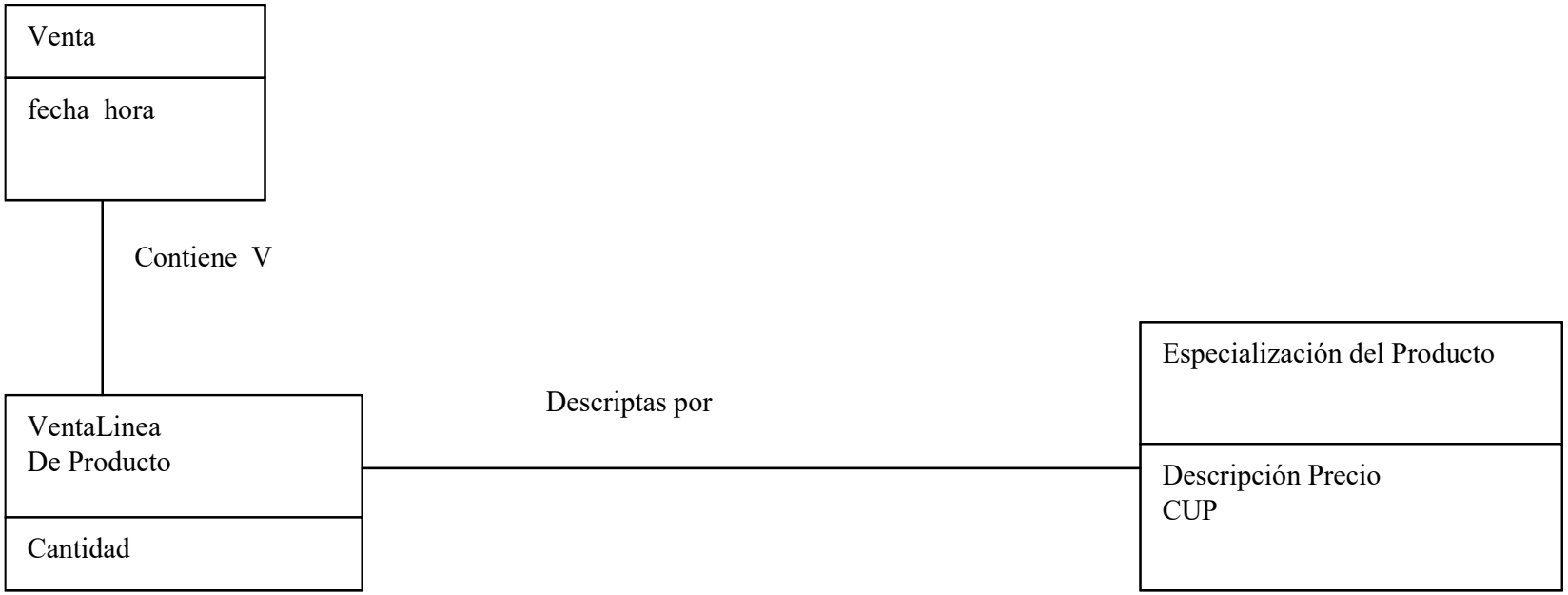
Patrones GRASP: "*Creador*"

Ejemplo (punto de ventas de Larman)

- En el ejemplo de punto de ventas. ¿Quién debería encargarse de crear una instancia Línea de Venta?.

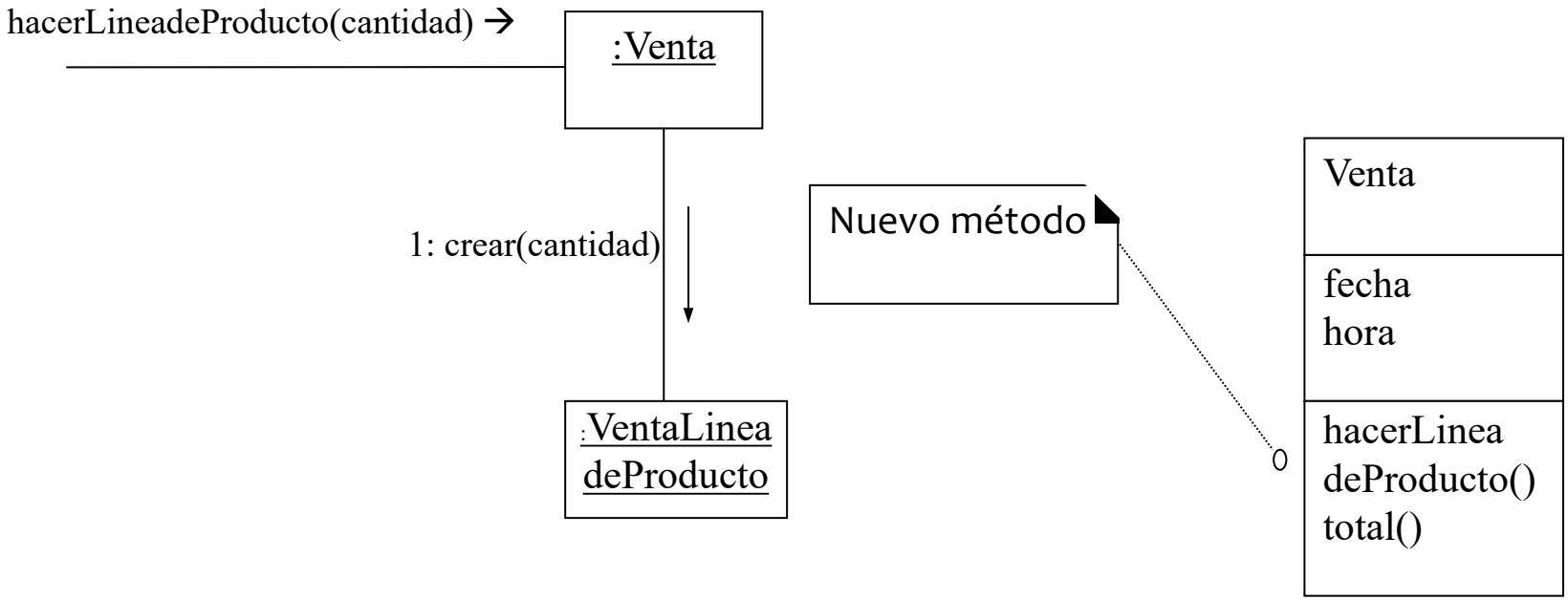
- Desde el punto de vista del patrón Creador, deberíamos buscar una clase que agregue, contenga y realice otras operaciones sobre este tipo de instancias.

Patrones GRASP: "*Creador*"



En este Modelo, una Venta contiene muchos objetos VLP; por ello el patrón ***Creador*** sugiere que ***Venta*** es idónea para **asumir la responsabilidad de crear la instancia VLP**, y a partir de aquí podemos diseñar las interacciones de los objetos en un diagrama de colaboración.

Patrones GRASP: "*Creador*"



Al asignar responsabilidades en un diagrama de interacción=> definimos en Venta un Método de ***"hacerlíneadeProducto"***.

Patrones GRASP: "*Creador*"

Explicación

- El propósito de este patrón es asignar responsabilidades relacionadas con la **creación de objetos producidos en cualquier evento**.
- El patrón **Creador** indica que una clase es idónea para asumir la responsabilidad de crear la **cosa contenida o registrada**.

Beneficios

- Facilita un **Bajo Acoplamiento**, => supone **menos dependencias** respecto al mantenimiento y mejores oportunidades de reutilización.

•NOTA: Un constructor por defecto es una subrutina cuya misión es inicializar un objeto de una clase sin parámetros, que no hace nada.
Sin embargo será invocado cada vez que se construya un objeto sin especificar ningún argumento, en cuyo caso el objeto será iniciado con los valores predeterminados por el sistema (los atributos numéricos a ceros, los alfanuméricos a nulos, y las referencias a objetos a null).

Patrones GRASP: "*Creador*"

Contraindicaciones

- *A menudo, la creación requiere un complejidad, por ejemplo utilizar instancias recicladas por motivo de rendimiento, crear instancias a partir de familias de clases basado en el valor de alguna propiedad externa,, etcétera. En este caso es aconsejable delegar la creación a una clase creadora.*

Patrones GRASP: "*Bajo Acoplamiento*"

Problema:

¿Cómo dar soporte a una dependencia escasa y a un aumento de la reutilización?

El acoplamiento es una medida de la fuerza con que una clase está conectada a otras clases, con que las conoce y con que recurre a ellas.

- Una clase con bajo (o débil) acoplamiento no depende de "muchas otras".
- Una clase con alto (o fuerte) acoplamiento (depende de "muchas otras") presentan los siguientes problemas:
  - Los cambios de las clases afines ocasionan cambios locales.
  - Son más difíciles de entender.
  - Son más difíciles de reutilizar porque se requiere la presencia de otras clases de las que dependen.

Solución:

Asignar una responsabilidad para mantener BAJA ACOPLAMIENTO

Patrones GRASP: "*Bajo Acoplamiento*"

EjemploSolución

Asignar responsabilidad para mantener bajo el acoplamiento.

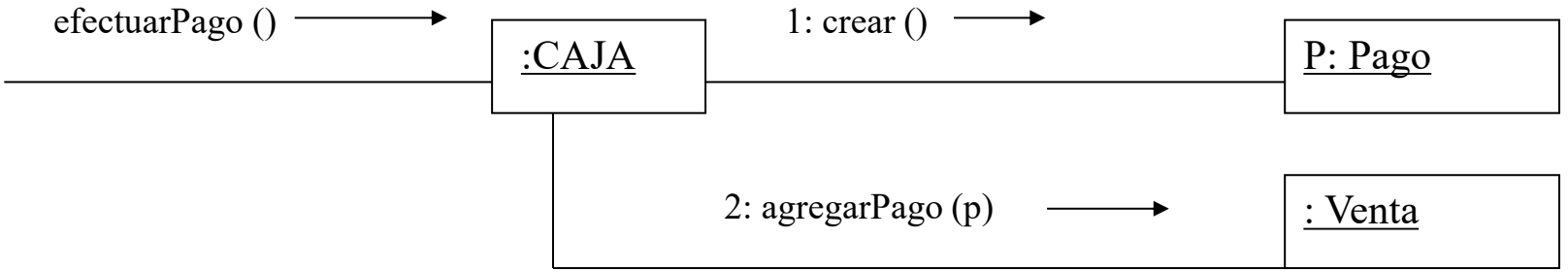


Suponga que necesitamos crear una instancia ***Pago*** y asociarla a ***Venta***. ¿Qué clase se encargará de hacer esto?

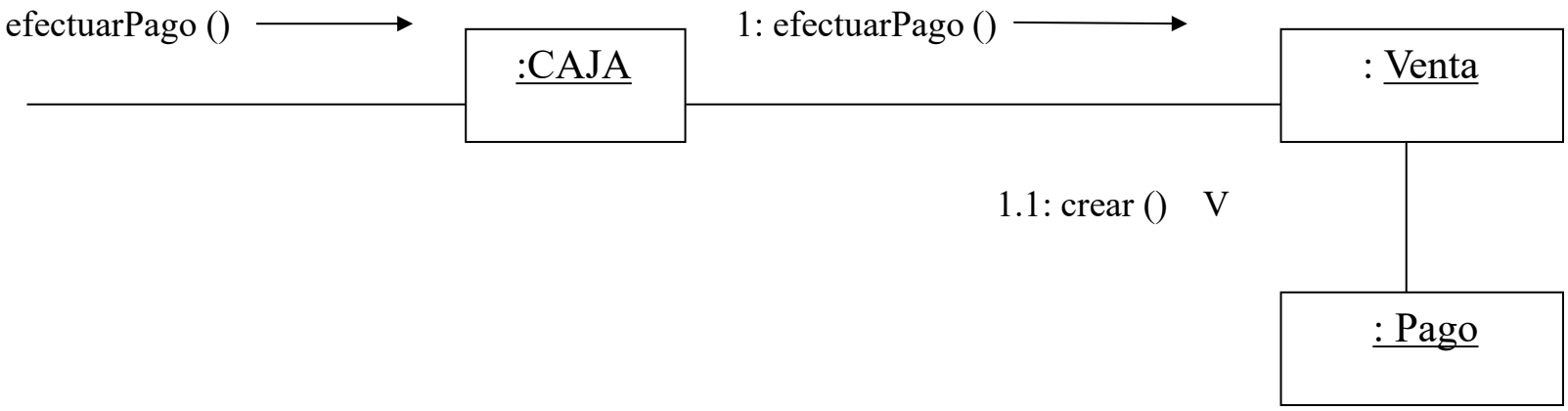
Como una instancia de *CAJA* **"registra"** un *Pago*, el patrón **Creador** indica que *CAJA* es un buen candidato para producir el *Pago*. La instancia *CAJA* podría entonces enviarle a *Venta* el mensaje: *agregarPago*, y con *Pago* como parámetro.

Patrones GRASP: "*Bajo Acoplamiento*"

Solución que acopla la clase CAJA



Solución Alternativa



Patrones GRASP: "*Bajo Acoplamiento*"

Explicación

- El Bajo Acoplamiento es un patrón **evaluativo** que el diseñador aplica al juzgar sus decisiones de diseño.
- El Bajo Acoplamiento soporta el diseño de clases **más independientes**, que reducen el **impacto de los cambios**, y también **más reutilizables**, que acrecientan la oportunidad de una mayor productividad.
- No puede considerarse en forma independiente como Experto o Alta Cohesión.
- En términos generales, han de tener escaso acoplamiento las clases muy genéricas y con grandes probabilidades de reutilización.

- El caso extremo de dicho patrón ´bajo acoplamiento´ **ocurre cuando existe poco o nulo acoplamiento**. **No conviene, pues se tendrá objetos que hacen de todo, complejos, o muy pasivos**

Patrones GRASP: "*Bajo Acoplamiento*"

Explicación

Acoplamiento de:

- **CONTENIDO** cuando un módulo referencia directamente al contenido de otro módulo.
- **COMUN**: cuando dos módulos acceden y afectan a un mismo valor global.
- **CONTROL**: cuando un módulo le envía a otro modulo un elemento de control que influye en la lógica

Patrones GRASP: "*Bajo Acoplamiento*"

- El bajo Acoplamiento soporta el diseño de clases que son mas independientes , lo que reduce el impacto del cambio.
- El bajo Acoplamiento no afectan los cambios en otros componentes

Patrones GRASP: "*Alta Cohesión*"

Problema

¿Cómo mantener la complejidad dentro de límites manejables?:

En el DOO, la **cohesión** es una medida de cuán **enfocados** están las **responsabilidades** de una **clase**.

Una alta cohesión caracteriza a las clases con responsabilidades estrechamente relacionadas para que no realicen un trabajo enorme.

A menudo representan un alto grado de abstracción o han asumido responsabilidades que deberían haber delegado.

Patrones GRASP: "*Alta Cohesión*"

Una clase con **baja cohesión** hace muchas cosas no afines o un trabajo excesivo.

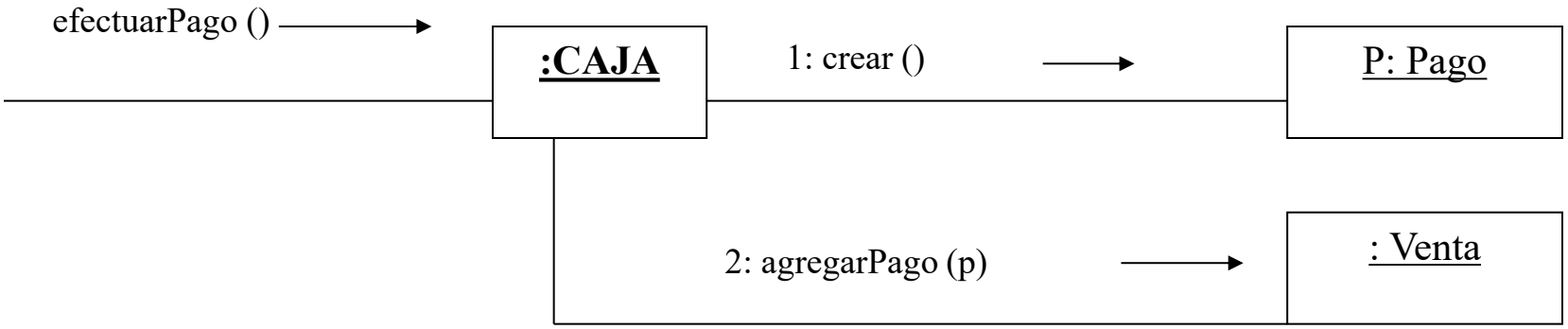
No conviene este tipo de clases, porque:

- Son difíciles de comprender.
- Son difíciles de reutilizar.
- Son difíciles de conservar.
- Son delicadas: las afectan constantemente los cambios.

Solución:

Asignar una responsabilidad de modo que la cohesión siga siendo alta.

Patrones GRASP: "*Alta Cohesión*"

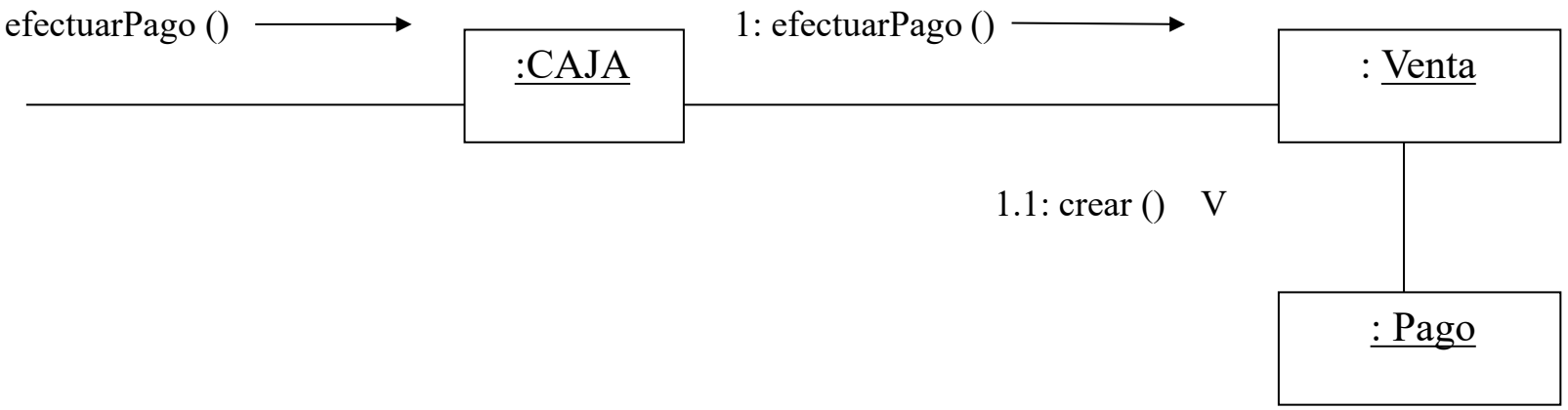


Esta asignación de responsabilidad coloca en la clase *CAJA* la generación del pago. (patrón ***Creador***).

Esto es aceptable. Pero si seguimos haciendo que la clase *CAJA* se encargue de efectuar algún trabajo con un número creciente de operaciones del sistema, se irá saturando con tareas y terminará por perder la cohesión.

En cambio, en el 2º ejemplo, se delega a la *Venta* la responsabilidad de ***crear*** el pago. Esto brinda un soporte a una **Alta Cohesión** y a un **Bajo Acoplamiento**.

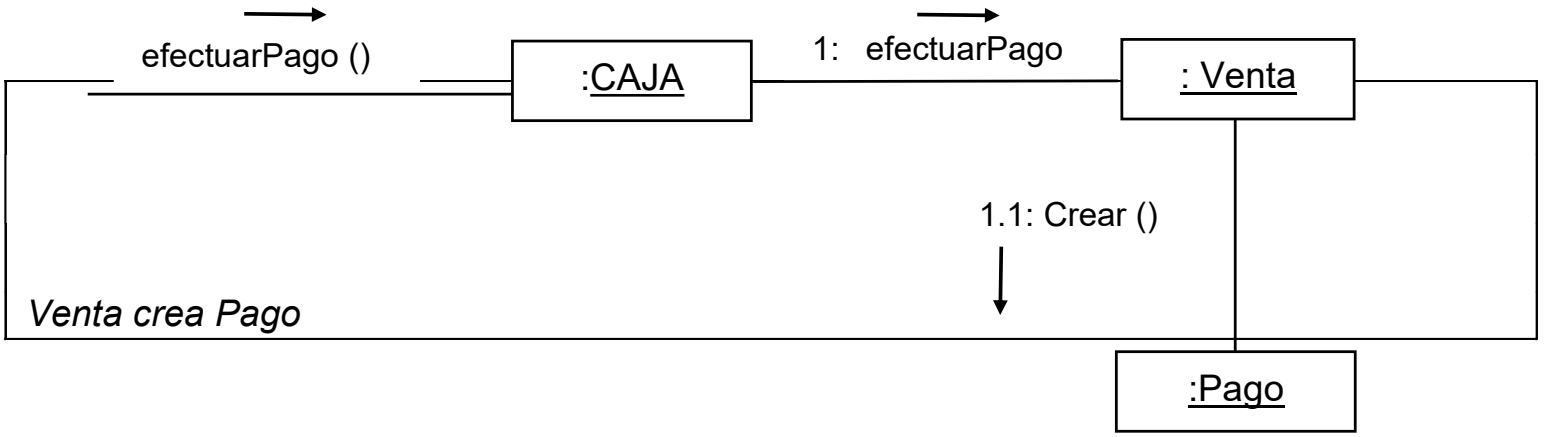
Patrones GRASP: "*Alta Cohesión*"



En cambio, en el 2º ejemplo, se delega a la *Venta* la responsabilidad de ***crear*** el pago. Esto brinda un soporte a una **Alta Cohesión** y a un **Bajo Acoplamiento**.

Patrones GRASP: "*Alta Cohesión*"

El nivel de cohesión no puede ser considerado independientemente del **Experto** y del **Bajo Acoplamiento**.



Patrones GRASP: "*Alta Cohesión*"

Grados de cohesión:

- **Muy baja Cohesión**: una **clase** es la **única responsable** de muchas cosas distintas en áreas funcionales diferentes (se deben dividir las responsabilidades en una familia de clases relacionadas).

- **Baja Cohesión**: una **clase** tiene la **responsabilidad exclusiva** de una **tarea compleja** dentro de una área funcional (dividir la clase en una familia de clases ligeras que compartan el trabajo).

Alta Cohesión: una **clase tiene responsabilidades moderadas** en un área y colabora con las otras para realizar las tareas (supuesto ideal

Patrones GRASP: "*Alta Cohesión*"

Grados de cohesión:

- **Cohesión Coincidentes.** Módulos múltiples tareas
- **Cohesión Lógica.** Módulo con muchas tareas, pero en ejecución solo se ejecuta una
- **Cohesión temporal,** Módulo con varias tareas ejecutadas al mismo momento
- **Cohesión de Procedimiento,** Módulo con muchas secuencia de pasos
- **Cohesión de Información,** trabajan sobre los mismos datos
- **Cohesión Funcional** Módulo que ejecuta una única tarea

Patrones GRASP: "*Alta Cohesión*"

Una **clase de Alta Cohesión** posee un número relativamente pequeño, con una importante funcionalidad relacionada y poco trabajo por hacer. **Colabora con otros objetos para compartir el esfuerzo si la tarea es grande**.

Una **clase** con mucha **Cohesión** es útil porque es bastante **fácil** darle **mantenimiento, entenderla y reutilizarla**.

El patrón Alta Cohesión presenta semejanzas con el mundo real. Sabemos que si alguien asume demasiadas responsabilidades, no será eficiente (hay que **delegar**).

Patrones GRASP: "*Alta Cohesión*"

Beneficios:

- Mejorar la claridad y la facilidad con que se entiende el diseño.
- Se simplifican el mantenimiento.
- A menudo se genera un Bajo Acoplamiento.
- Mayor capacidad de reutilización.

Patrones GRASP: “Diseño Modular”

- La modularidad es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos COHESIVOS y débilmente ACOPLADOS

Patrones GRASP: "*Controlador*"

Problema:

¿Quién debería encargarse de atender un evento del sistema?:

Un **evento del sistema** es un suceso generado por un actor externo (evento de entrada externo).

Se asocia a **operaciones del sistema** => respuestas a los **eventos del sistema**.

Por ejemplo: cuando un cajero oprime el botón ***"Terminar Venta"***, está generando un evento que indica que ***"la venta ha terminado"***.

Patrones GRASP: "*Controlador*"

Problema:

¿Quién debería ser el responsable de gestionar un evento de entrada al sistema?

Solución:

Asignar la **responsabilidad** del **manejo** de un mensaje de los **eventos** de un sistema **a una clase** que represente una de las siguientes opciones:

1) **El "sistema global" (controlador de fachada).**
2) **La empresa u organización global** (**controlador de fachada**).
3) **Algo en el mundo real que es activo** y que pueda participar en la tarea (**controlador de tareas**).
4) **Un manejador artificial** de todos los eventos del sistema de un caso de uso: ***"Manejador < NombreCasodeUso"*** (**controlador de casos de uso**).

Use la misma clase de controlador con todos los eventos del sistema en el mismo Caso de Uso.

Patrones GRASP: "*Controlador*"

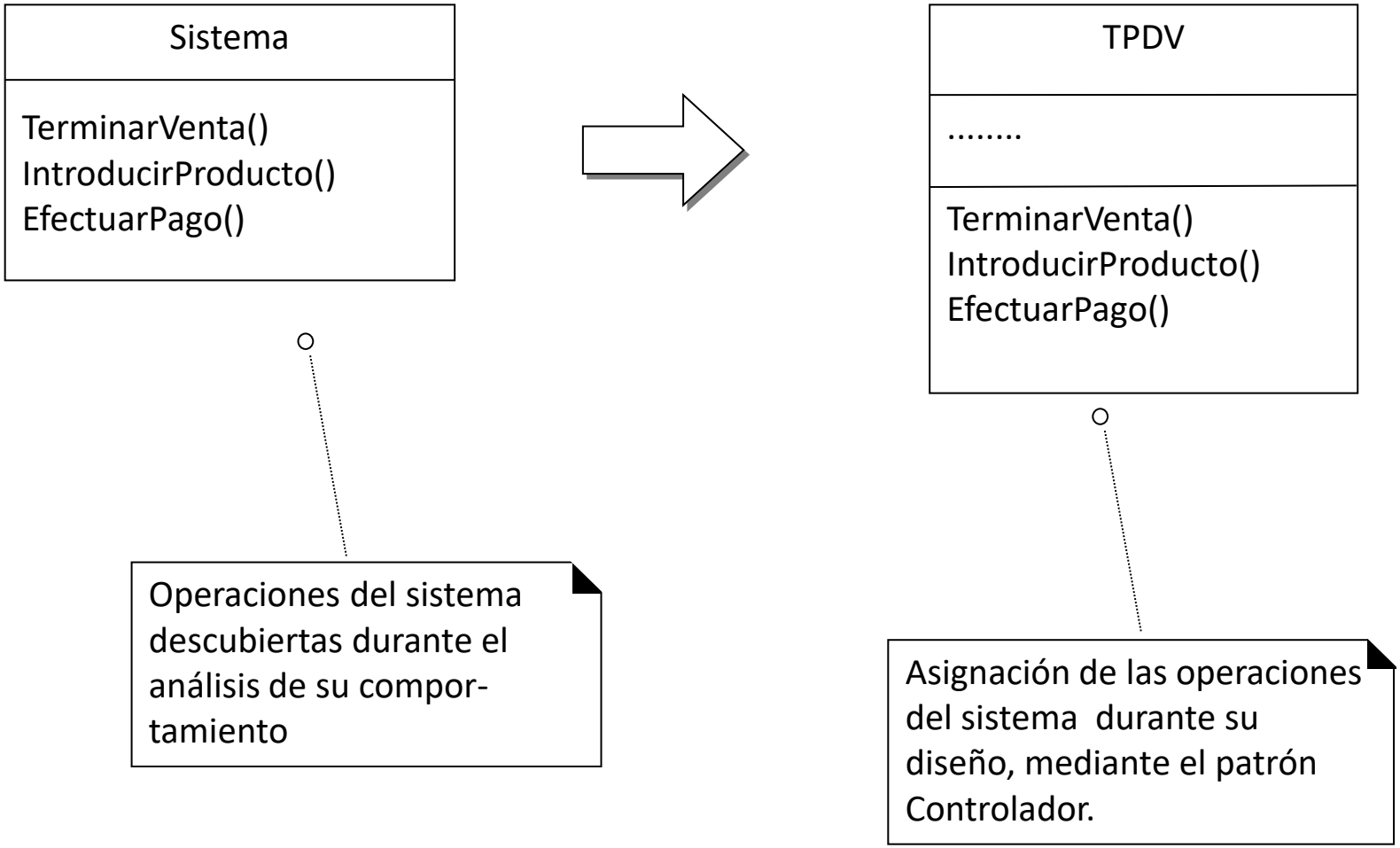
ejemplo:
¿Quién debería ser el responsable de gestionar un evento de entrada al sistema?

Sistema
finalizarVenta() IntroducirArtículo() crearNuevaVenta() realizarPago()

Durante el **análisis**, estas operaciones son asignadas al tipo **Sistema** para indicar que son operaciones del sistema. Pero ello no significa que una clase llamada Sistema las ejecute durante el diseño.

Durante el **diseño**, a la clase **Controlador** se le asigna la **responsabilidad** de las operaciones del **sistema**.

Patrones GRASP: "*Controlador*"



Patrones GRASP: "*Controlador*"

En **DOO** hay que **elegir los controladores** que manejen los eventos de entrada. Este patrón ofrece una guía para tomar decisiones apropiadas.
La misma clase controlador debería utilizarse con todos los eventos de **un Caso de Uso,** es decir **un controlador por CU** (recomendable**).**

Un defecto frecuente al diseñar controladores consiste en asignarles demasiadas responsabilidades. Normalmente, un controlador debería delegar a otros objetos el trabajo que ha de realizarse mientras coordina la actividad.

El **controlador de fachada** que representa al **"sistema"** global, es una clase que, para el diseñador, representa de alguna manera al sistema entero (unidad física: TPDV, Sistema, etc.). Son adecuados cuando el sistema sólo tiene pocos eventos o cuando es imposible redirigir los mensajes de los eventos del sistema a otros controladores.

Patrones GRASP: "*Controlador*"

En el "***Manejador artificial de Casos de Uso o* controlador para cada Caso de Uso"**. No es un objeto del dominio, sino un concepto artificial cuyo fin es dar soporte al sistema (una fabricación pura).
Por ejemplo, en el Caso de Uso ***ComprarProducto*** habrá una clase:

ManejadordeComprarProducto.

Es **conveniente** elegirlo, cuando un **controlador** comienza a "**saturarse**" con demasiadas responsabilidades. Este controlador constituye una buena alternativa cuando hay muchos eventos de sistemas entre varios procesos: asigna su manejo a clases individuales controlables.

Patrones GRASP: "*Controlador*"

Beneficio Principal:

Mayor potencial de los **componentes reutilizables**. Garantiza que los eventos del sistema sean manejados por la capa de los objetos del dominio y no por la de la interfaz.

Un diseño de interfaz como controlador reduce la posibilidad de reutilización, por estar ligado a una interfaz determinada que rara vez puede reutilizarse en otras aplicaciones.

En cambio, el hecho de **delegar en un controlador la responsabilidad de la operación de un sistema, entre las clases del dominio, permite una mejor reutilización de la lógica.**

Patrones GRASP: "*Controlador*"

Controladores Saturados:

Es una clase de controlador mal diseñada, presenta Baja Cohesión: está dispersa y tiene demasiadas áreas de responsabilidad.

Signos de Saturación:

- Una **sola** clase controlador recibe todos los eventos que son excesivos.
- El **controlador** realiza él mismo muchas tareas necesarias para cumplir el evento del sistema, sin que delegue trabajos (violación de los patrones Experto y Alto Acoplamiento).
- Un controlador posee muchos atributos y conserva información importante sobre el sistema que debería haber sido redistribuida entre objetos – y duplica la información.

Patrones GRASP: "*Controlador*"

- **Soluciones:**

- **Agregar mas controladores:** un sistema no necesariamente debe tener uno solamente. Además de los controladores de fachada, se recomienda emplear los controladores de papeles o los de Casos de Uso.

- Diseñe el controlador para que **delegue fundamentalmente** a otros objetos el desempeño de las responsabilidades de la operación del sistema.

Patrones GRASP: "*Controlador por CU y Manejador (Handler) por Clase:*"

Un controlador es responsable de coordinar la actividad, pero debe delegar la mayor parte del trabajo a otros objetos para evitar sobrecarga.

Controlador (Controller) de Fachada:
Funcionalidad: El controlador de fachada actúa como un punto de entrada principal para el sistema, su objetivo es coordinar grandes secciones de funcionalidad del sistema.

Controlador (Controller) por Caso de Uso (CU):
•**Funcionalidad**: El controlador es una clase que gestiona todo el flujo de un Caso de Uso. Coordina las interacciones necesarias para completar una tarea específica del sistema, delegando responsabilidades a otras clases.
•**Responsabilidad**: Su responsabilidad es más amplia, englobando la lógica de negocio necesaria para ejecutar todo el CU.
Por ejemplo, el controlador `ComprarProductoController` podría gestionar todo el proceso de compra, desde la selección del producto hasta el pago.
•**Abstracción**: Este controlador actúa como el intermediario principal que recibe las solicitudes de la interfaz de usuario y orquesta las respuestas necesarias, interactuando con varios objetos.

Patrones GRASP: "*Controlador y Manejador (Handler) por Clase:*"

Manejador (Handler) por Clase:

•**Funcionalidad**: El manejador es una clase o función que se encarga de una tarea específica o de manejar un evento dentro de una clase. Es más detallado y se enfoca en gestionar acciones particulares que forman parte de la funcionalidad general del sistema.

•**Responsabilidad**: La responsabilidad del manejador es más estrecha, como gestionar la validación de datos, procesar un evento de clic, o manejar una excepción. Cada clase puede tener uno o varios manejadores para distintos eventos o acciones.

•**Abstracción**: El manejador no tiene una visión completa del Caso de Uso, sino que se ocupa de acciones concretas y específicas. Por ejemplo, un `StockHandler` dentro de la clase `Producto` podría manejar la verificación de disponibilidad de stock.

•**entre Controladores:** En general, los controladores pueden comunicarse entre sí, si es necesario para completar una tarea compleja. Sin embargo, esto debe hacerse con cuidado para no romper la cohesión y aumentar **Interacción** (el acoplamiento) entre ellos.

Aclaracion: Patrones GRASP: “*Controlador y Manejador (Handler) por Clase:”*

Aclaraciòn:
En el examen, debido a limitaciones de tiempo, no vamos a poder implementar un controlador por cada Caso de Uso (CU). En su lugar, el diseño debe contemplar controladores por componente. Los componentes serán conjuntos de Casos de Uso que compartan una naturaleza o funcionalidad común.
Es importante que identifiquen y definan los componentes o módulos más relevantes del sistema. Esto les permitirá diseñar de manera modular, lo que es clave para mantener un sistema organizado y escalable.
Aunque el diseño de manejadores por clase es un aspecto importante, no se les solicitará en el examen. Sin embargo, tengan en cuenta que esto podría ser requerido en el Trabajo Práctico si así se lo indica.

Patrones GRASP: "*Controlador*"

MVC (Model–View –Controller)

- Origen a principios de 8Os para Smalltalk
- Divide la aplicación en 3 tipos de componentes: modelos, Vistas y controladores.

- ✓ **Modelo**: principal funcionalidad e información
- ✓ **Vista**: muestra información al usuario
- ✓ **Controlador**: controla los inputs del usuario

Patrones GRASP: "*Controlador*" y MVC

El MVC se basa en un concepto introducido en el año de 1970, por Dijkstra, el concepto de ***"separación de incumbencias"*** *en OO*

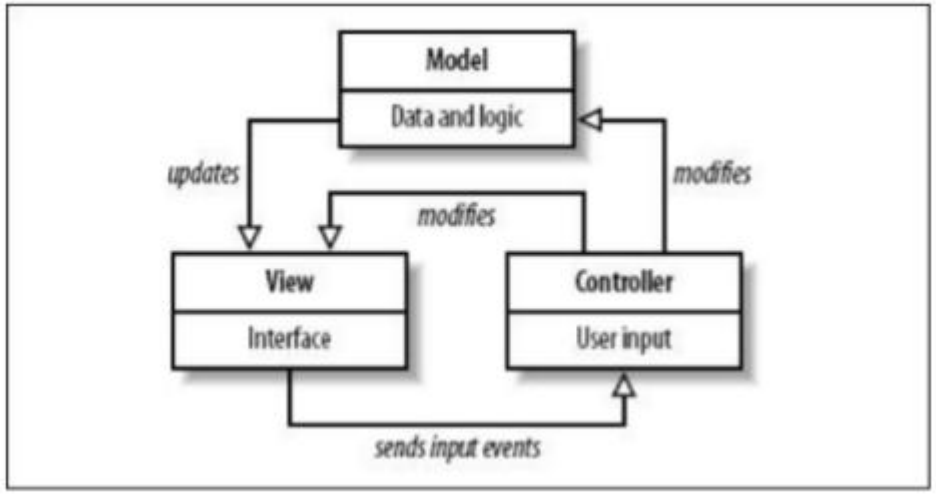
Implica que la solución de problema, involucra varias incumbencias o asuntos, los que deben ser **identificadas** y analizados en forma **independiente**.

Separando las incumbencias, se disminuye la complejidad.
Se puede cumplir con requerimientos relacionados con la calidad.

Las técnicas de modelado que se usan en la etapa de diseño de un sistema se basan en partirlo en varios subsistemas que resuelvan parte del problema o correspondan a una parte del dominio sobre el que trata.

Patrones GRASP: "*Controlador*"

MVC (Model–View –Controller)



Patrones GRASP: "*Controlador*" y MVC

El **Modelo** es el responsable de:

- Acceder a la capa de almacenamiento de datos. Lo ideal es que el modelo sea independiente del sistema de almacenamiento.
- Define las reglas de negocio (la funcionalidad del sistema). Un ejemplo de regla puede ser: "Si la mercancía pedida no está en el almacén, consultar el tiempo de entrega estándar del proveedor".
- Lleva un registro de las vistas y controladores del sistema.

  Si estamos ante un modelo activo, notificará a las vistas los cambios que en los datos pueda producir un agente externo (por ejemplo, un fichero batch que actualiza los datos, un temporizador que desencadena una inserción, etc.).

  Un ejemplo de MVC con un modelo pasivo (aquel que no notifica cambios en los datos) es la navegación web, que responde a las entradas del usuario, pero no detecta los cambios en datos del servidor.

Patrones GRASP: "*Controlador*" y MVC

Vista
La vista es la capa de la aplicación que ve el usuario en un formato adecuado para interactuar, es nuestra interfaz grafica.
Las **vistas** son responsables de:

- Recibir datos del modelo y los muestra al usuario.
- Tienen un registro de su controlador asociado (normalmente porque además lo instancia).
- Pueden dar el servicio de "Actualización()", para que sea invocado por el controlador o por el modelo (cuando es un modelo activo que informa de los cambios en los datos producidos por otros agentes).

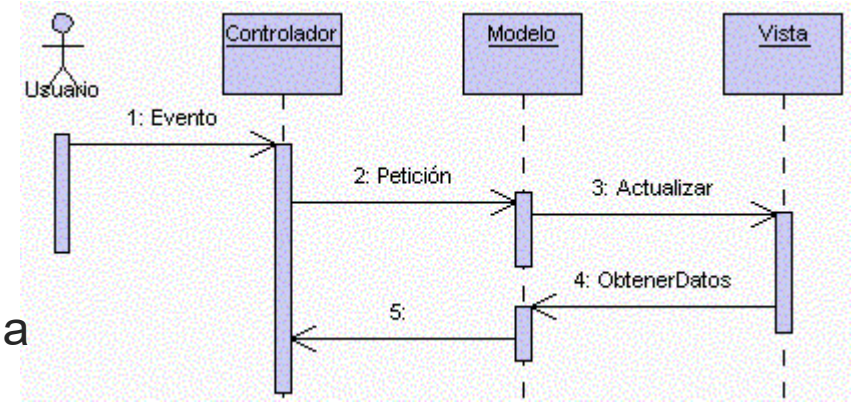
Patrones GRASP: "*Controlador*" y MVC

Diag de secuencia

Pasos:

1. El usuario introduce el **evento**.
2. El Controlador recibe el evento y lo traduce en una **petición** al **Modelo** (aunque también puede llamar directamente a la vista).
3. El **modelo** (si es necesario) llama a la vista para su **actualización**.
4. Para cumplir con la **actualización** la Vista puede **solicitar** datos al **Modelo**.
5. El Controlador recibe el control.

NOTA: algunos MVC, el modelo actualiza directamente a la vista



Otros modelos

- **MVC: Desacopla la lógica de negocio, la presentación y la interacción.**
- **MVVM: Mayor enfoque en el binding de datos y desacoplamiento de la vista y el modelo.**
- **MVP: Separación más fuerte de la lógica de presentación, especialmente útil para pruebas unitarias.**
- **Flux/Redux: Arquitectura de flujo unidireccional para mantener un estado centralizado y predecible.**
- **Clean Architecture: Enfoque más estricto en capas independientes, mejor para aplicaciones grandes.**
- **EDA: Enfoque basado en eventos, altamente escalable y flexible.**
- **Microkernel: Modularidad y extensibilidad mediante plug-ins.**

Otros Patrones GRASP

- Diseño Modular
- Polimorfismo
- Fabrica Pura
- Indireccion
- Variaciones Protegidas

Patrones GRASP: “Diseño Modular”

- La modularidad es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos COHESIVOS y débilmente ACOPLADOS

Patrones GRASP: “Polimorfismo”

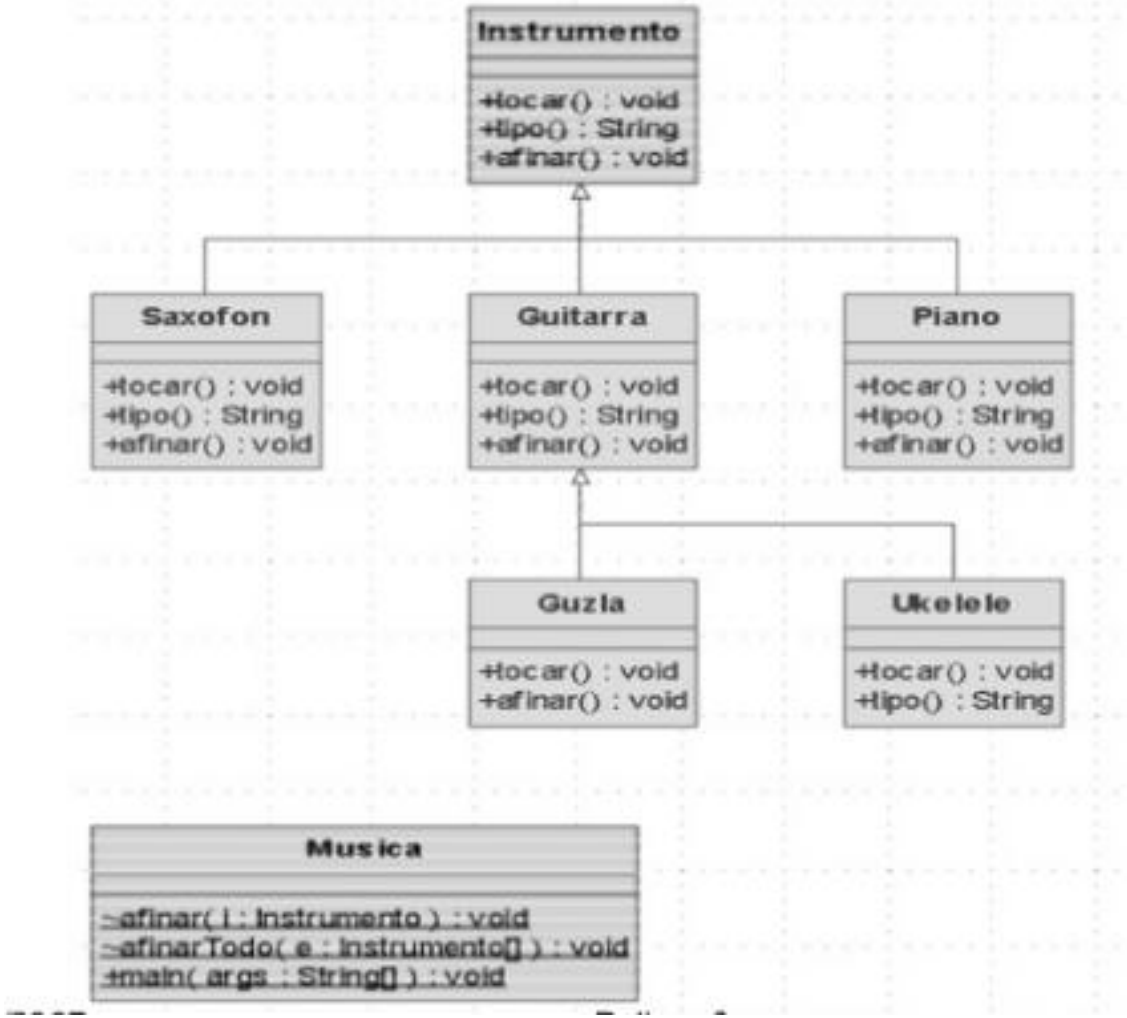
- Tiene varios significativos, en este contexto significa **Asignar el mismo Nombre a servicios diferentes**
- En POO, el polimorfismo se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos

Patrones GRASP: “Polimorfismo”

3 FORMAS DE POLIMORFISMO

- Sobre-escritura de métodos
  - Métodos de una subclase sobreescriben los métodos de la superclase
- Sobre-escritura de métodos (impl) abstractos
  - Métodos de la subclase implementan los métodos abstractos de la superclase abstracta (clase abstracta)
- Sobre-escritura de métodos (impl) a través de interface
  - Métodos de una clase concreta implementan los métodos de la interface

Patrones GRASP: “Polimorfismo”



Patrones GRASP: “Fabricacion Pura”

- **Asignar** un conjunto de **responsabilidades** altamente cohesivo a **una clase artificial** o de conveniencia que no representa un concepto del dominio del problema.
- Tal clase es una fabricación de la imaginación.

2Ejemplo:

- La clase venta tiene que almacenar objetos en la **BD**, colocar esta responsabilidad a la clase Venta genera que la reutilización va ser muy pobre , porque muchas clases van a requerir almacenar objetos, por lo tanto una solución razonable es crear una nueva clase que sea responsable de almacenar objetos en algún almacenamiento (por ejemplo, alguna BD). Esta clase es una invención de la imaginación

Patrones GRASP: “Indirecciones”

- A veces se necesita evitar el acoplamiento directo entre dos objetos, para desacoplar los objetos se utiliza un intermediario (crea una indirección entre los objetos)

- Ejemplo:

La clase pago tiene que comunicar a otro sistema de “impuesto” que el pago se realizo, se podría añadir una clase como un Adaptador que es una clase cuyo objetivo es ser intermediario.

Patrones GRASP: “Variaciones Protegidas”

- Identificar puntos de inestabilidad en el modelo

- Por ejemplo:
- Un **sistema debe integrarse con varios sistemas** , se crea una **clase** para manejar la integración con los otros sistemas.
- Similar a indirecciones